

ChargeHub Technician Module

Run guide, database relationships, routes, and required integration work

What was implemented

The technician feature is database-backed with TypeORM. The registration record is the **technician_profiles** table; it stores the information a future login/authentication feature will use to identify a technician. New profiles start with approvalStatus = Pending, so a technician cannot record work until an Admin approves the profile.

Table	Purpose and relationship
technician_profiles	Technician registration/profile. Has many maintenance tasks, safety checks, and fault reports; has many specialisations.
maintenance_tasks	Charging or hardware work logged by one technician. Many tasks belong to one profile.
safety_checks	Completed station safety inspections. Many checks belong to one profile.
fault_reports	Reported and resolved station faults. Many faults belong to one profile.
specialisations	Reusable skill labels. Many technicians can have many specialisations through technician_profile_specialisations.

Relationship requirements met

1. One-to-many: one technician profile -> many maintenance tasks, safety checks, and fault reports. 2. Many-to-many: technician profiles <-> specialisations. The maintenance create, read, update, and delete endpoints demonstrate CRUD against the one-to-many relationship.

How to run

1. Ensure PostgreSQL is running and create the database configured by the application. The current AppModule configuration uses PostgreSQL on localhost:5432 with database EV_Charging_Network.

```
CREATE DATABASE "EV_Charging_Network";
```

2. From the project root, install dependencies and start the NestJS server.

```
npm install npm run start:dev
```

3. With synchronize: true enabled in development, TypeORM creates the technician tables at startup. Use an API tool such as Postman at http://localhost:3000.

Important workflow

POST /technician creates a Pending technician profile. An Admin module should approve the profile before the profile can create maintenance tasks, safety checks, or fault reports. The technician module correctly blocks those work actions while Pending or Rejected.

Example: create a technician profile

```
POST /technician { "fullName": "Amina Rahman", "email": "amina@example.com", "phone":  
"01700000000", "location": "Dhaka", "experience": 3, "primarySpecialisation": "Electrical",  
"specialisations": ["Electrical", "Battery Systems"], "country": "Bangladesh" }
```

Example: create a maintenance task after admin approval

```
POST /technician/1/maintenance { "category": "Charging", "stationId": "ST-001", "issue":  
"Socket overheating", "notes": "Inspect the connector", "maintenanceDate": "2026-08-03" }
```

Technician routes

Method	Route	Use
POST	/technician	Register technician profile
GET	/technician/:id	Read profile with relationships
PATCH	/technician/:id/country	Update profile country
GET	/technician/by-date/search?joiningDate=YYYY-MM-DD	Find profiles joined that day
GET	/technician/country/unknown	Find profiles with Unknown country
POST / GET	/technician/:technicianId/maintenance	Create or list maintenance tasks
PATCH / DELETE	/technician/maintenance/:id	Update or delete maintenance task
POST	/technician/:technicianId/specialisations/:name	Add many-to-many specialisation
POST / GET	/technician/:technicianId/safety-checks ; /technician/safety-checks/:id	Create or read safety check
POST / PATCH	/technician/:technicianId/faults ; /technician/faults/:id	Create fault or update resolution

Required changes outside the technician module

No non-technician source file was changed for this delivery. To fully satisfy the APW marks distribution, make these separate integration changes with the owner of the relevant module:

- 1. Database bootstrap:** Keep TypeOrmModule.forRoot with PostgreSQL and autoLoadEntities: true in AppModule, preferably reading credentials from environment variables. This is already present in the working project but remains outside the technician module.
- 2. Admin approval:** Add an Admin-only endpoint that changes technician_profiles.approvalStatus to Approved or Rejected. The technician service already enforces this status before work can be logged.
- 3. Authentication:** Add bcrypt and JWT dependencies, a passwordHash column flow, sign-in endpoint, JWT guard, and role/approval guard. The provided project does not currently have @nestjs/jwt or bcrypt installed, and the requested technician-only scope prevents modifying package.json or global application setup.
- 4. Production:** Replace synchronize: true with migrations and put database/JWT secrets in .env. Do not use the hard-coded development database password in production.

Mailer was intentionally not included, as requested.